



# **UNIVERSITY OF BUEA**

**FACULTY OF ENGINEERING AND TECHNOLOGY**

**DEPARTMENT OF COMPUTER ENGINEERING**

**CEF440: INTERNET AND MOBILE PROGRAMMING**

## **DESIGN AND IMPLEMENTATION OF A MOBILE APPLICATION FOR CAR FAULT DIAGNOSIS (CARDIAG)**

*Course Project Final Report Group 12*

**Submitted by: Group 12**

**AWA PRECIOUS NKENG FE23A019**

**GUIMAZEU KENFACK DANIELLE FE23A067**

**NGANE IDRIS AJANG FE23A110**

**TACHINDA NJIGONG TRESOR FE23A147**

**TAMBE ENOWTABI PRINCELY FE23A155**

**Course Instructor: Dr. Valery Nkemeni**

**Academic Year: 2025/2026**

**Second Semester, Level 400**

## **DEDICATION**

This work is dedicated to God Almighty, for the grace, strength, and guidance granted throughout the duration of this project.

It is also dedicated to our course instructor, Dr. Valery Nkemeni, for his patient supervision, technical direction, and continuous encouragement throughout the semester.

Finally, this work is dedicated to all the members of Group 12, in recognition of the hard work, collaboration, and commitment each member brought to every task that made this project possible.

## **ACKNOWLEDGEMENT**

We wish to express our sincere gratitude to God Almighty for the wisdom, health, and perseverance that carried us through the successive stages of this project.

We are deeply grateful to our course instructor, Dr. Valery Nkemeni, for his guidance throughout the CEF440 Internet and Mobile Programming course. His feedback at every stage of the project, from requirement gathering through to database implementation, shaped the quality of the work presented in this report.

We also thank the Department of Computer Engineering and the Faculty of Engineering and Technology, University of Buea, for providing the academic framework and resources that made this project-based learning experience possible.

Our appreciation extends to the car owners, mechanics, and other respondents who took part in our surveys and interviews during the requirement gathering phase, and whose input shaped the direction of the CarDiag application.

Finally, we acknowledge the effort, patience, and cooperation of every member of Group 12. This report is the product of a genuinely collective effort spanning seven successive tasks, and each member's contribution was essential to its completion.

## ABSTRACT

Access to reliable vehicle fault diagnosis remains a significant challenge for drivers in Cameroon, where formal diagnostic equipment is scarce, mechanics are not always trustworthy or affordable, and language barriers frequently complicate communication between drivers and technicians. This report presents CarDiag, an artificial-intelligence-based mobile application developed by Group 12 for the CEF440 Internet and Mobile Programming course at the University of Buea, designed to help vehicle owners identify car faults by scanning dashboard warning lights, recording engine sounds, or describing problems in text, and to receive plain-language explanations, urgency ratings, repair guidance, and relevant video tutorials in return.

The project was carried out across seven successive tasks. It began with a study of mobile application types, programming languages, frameworks, architectural patterns, requirement engineering practice, and cost estimation techniques. This was followed by a structured requirement gathering exercise involving an eighty-responder survey, stakeholder interviews, brainstorming, reverse engineering of competing applications, and data collection for the underlying fault and audio datasets. The gathered requirements were then analyzed for completeness, clarity, feasibility, and prioritized using the MoSCoW method, culminating in a Software Requirements Specification. System modelling followed, producing a context diagram, a data flow diagram, use case diagrams and specifications, sequence diagrams, a class diagram, and a deployment diagram that together define the structure and behavior of the system.

The application's identity and interface were then defined, covering its technical identifiers, color palette, typography, logo, and ten core screens, implemented as a cross-platform React Native frontend. A PostgreSQL database extended with the pgvector extension was designed and implemented to store a catalogue of over six thousand OBD-II fault codes together with their vector embeddings, supporting an artificial-intelligence-driven similarity search pipeline built on a layered Spring Boot backend. The report concludes by drawing these seven tasks together into a single account of the system that was built, the decisions that shaped it, and the results achieved.

The resulting system demonstrates that a locally grounded, bilingual, offline-capable diagnostic tool is both technically feasible and strongly desired by its intended users, and it provides a solid foundation for continued development beyond the scope of this course.

**Keywords:** *mobile application development, vehicle fault diagnosis, artificial intelligence, requirement engineering, system modelling, React Native, PostgreSQL, pgvector, Spring Boot, Cameroon.*

# TABLE OF CONTENTS

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| <b>DEDICATION .....</b>                                         | <b>ii</b>  |
| <b>ACKNOWLEDGEMENT .....</b>                                    | <b>iii</b> |
| <b>ABSTRACT .....</b>                                           | <b>iv</b>  |
| <b>LIST OF TABLES .....</b>                                     | <b>ix</b>  |
| <b>LIST OF FIGURES .....</b>                                    | <b>xi</b>  |
| <b>LIST OF ABBREVIATIONS.....</b>                               | <b>xii</b> |
| <b>1. MOBILE APP DEVELOPMENT PROCESS.....</b>                   | <b>1</b>   |
| 1.1 Introduction .....                                          | 1          |
| 1.2 Types of Mobile Applications .....                          | 1          |
| 1.3 Mobile App Programming Languages and Frameworks.....        | 2          |
| 1.4 Mobile App Architectures and Design Patterns .....          | 2          |
| 1.5 Requirement Engineering Process .....                       | 3          |
| 1.6 Mobile App Development Cost Estimation.....                 | 4          |
| 1.7 Chapter Summary .....                                       | 5          |
| <b>2. REQUIREMENT GATHERING .....</b>                           | <b>6</b>   |
| 2.1 Introduction .....                                          | 6          |
| 2.2 Stakeholder Identification.....                             | 6          |
| 2.3 Requirement Gathering Techniques .....                      | 7          |
| 2.4 Data Gathering and Cleaning .....                           | 7          |
| 2.5 User Reluctance Assessment .....                            | 8          |
| 2.6 Consolidated High-Level Requirements .....                  | 9          |
| <b>3. REQUIREMENT ANALYSIS .....</b>                            | <b>11</b>  |
| 3.1 Introduction .....                                          | 11         |
| 3.2 Completeness, Clarity, and Feasibility .....                | 11         |
| 3.3 Inconsistencies, Ambiguities, and Missing Information ..... | 11         |
| 3.4 Requirement Prioritisation (MoSCoW).....                    | 12         |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| 3.5 Requirement Classification.....                         | 12        |
| 3.6 Software Requirements Specification (SRS) Summary ..... | 13        |
| 3.7 System Interfaces and Error Handling.....               | 14        |
| 3.8 Stakeholder Validation .....                            | 15        |
| <b>4. SYSTEM MODELLING AND DESIGN .....</b>                 | <b>16</b> |
| 4.1 Introduction .....                                      | 16        |
| 4.2 Context Diagram.....                                    | 16        |
| 4.3 Data Flow Diagram .....                                 | 17        |
| 4.4 Use Case Diagram .....                                  | 18        |
| 4.5 Sequence Diagrams .....                                 | 21        |
| 4.6 Class Diagram.....                                      | 22        |
| 4.7 Deployment Diagram .....                                | 23        |
| <b>5. UI DESIGN AND IMPLEMENTATION .....</b>                | <b>25</b> |
| 5.1 Introduction .....                                      | 25        |
| 5.2 Application Identity .....                              | 25        |
| 5.3 Design Objectives and Principles .....                  | 26        |
| 5.4 Colour Palette and Typography .....                     | 26        |
| 5.5 Navigation and Components .....                         | 27        |
| 5.6 Screen Design .....                                     | 28        |
| 5.7 Frontend Implementation .....                           | 29        |
| 5.8 Testing .....                                           | 30        |
| <b>6. DATABASE DESIGN AND IMPLEMENTATION .....</b>          | <b>32</b> |
| 6.1 Introduction .....                                      | 32        |
| 6.2 Data Elements.....                                      | 32        |
| 6.3 Conceptual Design.....                                  | 33        |
| 6.4 Entity Relationship Diagram .....                       | 34        |
| 6.5 Database Implementation .....                           | 34        |
| 6.6 Backend Implementation.....                             | 35        |
| 6.7 Connecting the Database to the Backend .....            | 35        |

|                                            |           |
|--------------------------------------------|-----------|
| 6.8 Chapter Summary .....                  | 36        |
| <b>7. FINAL PROJECT PRESENTATION .....</b> | <b>37</b> |
| 7.1 Introduction .....                     | 37        |
| 7.2 Project Overview .....                 | 37        |
| 7.3 Demonstration Walkthrough .....        | 37        |
| 7.4 How the Seven Tasks Fit Together.....  | 38        |
| 7.5 Achievements .....                     | 39        |
| 7.6 Limitations and Future Work .....      | 40        |
| 7.7 Conclusion .....                       | 40        |
| <b>REFERENCES .....</b>                    | <b>41</b> |



## LIST OF TABLES

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| Table 1.1 Structural comparison of the three mobile application categories. ....     | 1  |
| Table 1.2 Comparison of candidate languages and frameworks.....                      | 2  |
| Table 1.3 Architectural patterns summarized by principle and context.....            | 3  |
| Table 1.4 Feature complexity bands and indicative hour ranges. ....                  | 4  |
| Table 1.5 Indicative hourly rates by developer seniority and region. ....            | 5  |
| Table 1.6 Requirement elicitation techniques reviewed for this project.....          | 4  |
| Table 2.1 Stakeholders identified for the CarDiag project.....                       | 7  |
| Table 2.2 Summary of raw versus cleaned dataset volumes.....                         | 8  |
| Table 2.3 Reluctance factors and their planned mitigations. ....                     | 9  |
| Table 2.4 Consolidated high-level functional and non-functional requirements. ....   | 10 |
| Table 2.5 Data sources used to build the image and audio training datasets. ....     | 8  |
| Table 3.1 Inconsistencies identified during requirement analysis. ....               | 12 |
| Table 3.2 Requirement prioritization using the MoSCoW method. ....                   | 12 |
| Table 3.3 Non-functional requirements by category.....                               | 13 |
| Table 3.4 System interfaces and their defined error-handling behavior. ....          | 14 |
| Table 3.5 Stakeholder validation summary.....                                        | 15 |
| Table 4.1 Data stores supporting the six internal processes of the DFD. ....         | 18 |
| Table 4.2 Summary of the principal CarDiag use cases. ....                           | 20 |
| Table 4.3 Actors identified in the CarDiag use case model. ....                      | 20 |
| Table 5.1 CarDiag color palette and usage. ....                                      | 27 |
| Table 5.2 Development environment requirements for the frontend. ....                | 30 |
| Table 5.3 The five tabs of the CarDiag bottom navigation bar.....                    | 27 |
| Table 5.4 Severity badge color coding used across diagnosis and history screens..... | 28 |
| Table 6.1 Sample of the Fault data elements. ....                                    | 32 |
| Table 6.2 Entity relationships in the conceptual data model.....                     | 33 |
| Table 6.3 Database management system choices and rationale.....                      | 35 |

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| Table 6.4 Backend layer responsibilities in the CarDiag Spring Boot application..... | 35 |
| Table 7.1 Contribution of each task to the finished CarDiag system.....              | 39 |
| Table 7.2 CEF440 project schedule and mark allocation by task. ....                  | 39 |

## LIST OF FIGURES

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Figure 4.1 Context diagram of the CarDiag system. ....                              | 16 |
| Figure 4.2 Level 1 data flow diagram of the CarDiag system. ....                    | 17 |
| Figure 4.3 Main system use case diagram for CarDiag. ....                           | 19 |
| Figure 4.4 Sequence diagram for the dashboard warning-light scanning use case. .... | 21 |
| Figure 4.5 Sequence diagram for the offline diagnosis fallback. ....                | 22 |
| Figure 4.6 Class diagram of the CarDiag domain model. ....                          | 23 |
| Figure 4.7 Deployment diagram of the CarDiag system. ....                           | 24 |
| Figure 5.1 The CarDiag logo. ....                                                   | 25 |
| Figure 5.2 The CarDiag app icon. ....                                               | 26 |
| Figure 5.3 High-fidelity mockups of the ten core CarDiag screens. ....              | 29 |
| Figure 5.4 Excerpt of the registration screen implementation in React Native. ....  | 30 |
| Figure 6.1 Entity relationship diagram of the CarDiag database. ....                | 34 |

## LIST OF ABBREVIATIONS

| Abbreviation | Meaning                                                                |
|--------------|------------------------------------------------------------------------|
| AI           | Artificial Intelligence                                                |
| API          | Application Programming Interface                                      |
| CAN          | Controller Area Network                                                |
| DFD          | Data Flow Diagram                                                      |
| DTC          | Diagnostic Trouble Code                                                |
| ER           | Entity Relationship                                                    |
| FR           | Functional Requirement                                                 |
| GPS          | Global Positioning System                                              |
| HTTPS        | Hypertext Transfer Protocol Secure                                     |
| JSON         | JavaScript Object Notation                                             |
| MoSCoW       | Must have, Should have, Could have, Won't have (prioritization method) |
| MVC          | Model–View–Controller                                                  |
| MVP          | Model–View–Presenter                                                   |
| MVVM         | Model–View–View Model                                                  |
| NFR          | Non-Functional Requirement                                             |
| OBD-II       | On-Board Diagnostics, version II                                       |
| PWA          | Progressive Web App                                                    |
| REST         | Representational State Transfer                                        |
| SDK          | Software Development Kit                                               |
| SQL          | Structured Query Language                                              |
| SRS          | Software Requirements Specification                                    |
| UI           | User Interface                                                         |
| UML          | Unified Modeling Language                                              |
| UX           | User Experience                                                        |

# 1. MOBILE APP DEVELOPMENT PROCESS

## 1.1 Introduction

Before any implementation work began on CarDiag, the team first studied the landscape of mobile application development: the structural categories of mobile applications, the programming languages and frameworks available, the architectural patterns that govern code organization, the requirement engineering process, and the methods used to estimate project cost. This chapter summarizes that study and the technology decisions it informed for the remainder of the project.

## 1.2 Types of Mobile Applications

Mobile applications fall into three structural categories: native applications, compiled specifically for one operating system and offering the highest performance and full device API access; Progressive Web Apps (PWAs), browser-based applications offering instant, store-free updates but limited hardware access; and hybrid applications, which package web technologies inside a native shell to combine store distribution with a shared codebase. Table 1.1 compares the three categories.

| Feature            | Native                    | Progressive Web App     | Hybrid              |
|--------------------|---------------------------|-------------------------|---------------------|
| User experience    | Seamless, platform-native | Good, browser-dependent | Solid, some latency |
| Development cost   | High (two codebases)      | Low (single codebase)   | Medium              |
| Distribution       | App stores                | Web browser / URL       | App stores          |
| Offline capability | Excellent                 | Limited                 | Good                |
| Hardware access    | Full                      | Limited by browser      | Partial via plugins |

*Table 1.1 Structural comparison of the three mobile application categories.*

Because CarDiag requires reliable camera, microphone, Bluetooth (for optional OBD-II adapters), and offline access, a Progressive Web App was ruled out at this stage. The choice therefore narrowed to native or hybrid/cross-platform development, discussed further in Section 1.4.

### 1.3 Mobile App Programming Languages and Frameworks

Native development relies on platform-specific languages like Kotlin and Java for Android, Swift and Objective-C for iOS, each requiring separate specialist teams. Cross-platform languages such as Dart (Flutter), JavaScript/TypeScript (React Native, Ionic), and C# (.NET MAUI) allow a single codebase to target both Android and iOS. Table 1.2 summarizes the main options considered.

| Language / Framework                 | Type             | Performance | Code sharing         |
|--------------------------------------|------------------|-------------|----------------------|
| Kotlin / Jetpack Compose             | Native (Android) | Highest     | Android only         |
| Swift / SwiftUI                      | Native (iOS)     | Highest     | Apple platforms only |
| Dart / Flutter                       | Cross-platform   | High        | ~95%                 |
| JavaScript-TypeScript / React Native | Cross-platform   | Medium–High | ~85%                 |
| C# / .NET MAUI                       | Cross-platform   | Medium–High | ~80%                 |
| HTML-CSS-JS / Ionic + Capacitor      | Hybrid           | Medium      | ~90%+                |

*Table 1.2 Comparison of candidate languages and frameworks.*

React Native was selected for CarDiag. The deciding factors were: a single JavaScript/TypeScript codebase covering both Android and iOS, a very large ecosystem of libraries relevant to camera, audio, and Bluetooth access, and the team's existing familiarity with JavaScript from other coursework. The React Native CLI (rather than Expo) was used specifically because it grants full native project access, which is required for background microphone recording.

### 1.4 Mobile App Architectures and Design Patterns

Six architectural patterns were reviewed: MVC, MVP, MVVM, Clean Architecture, BLoC, and Redux. Each imposes a different discipline on where logic lives and how components communicate, trading off simplicity against testability and long-term maintainability. Table 1.3 summarizes the comparison.

| Pattern            | Key principle                                      | Best suited for                   |
|--------------------|----------------------------------------------------|-----------------------------------|
| MVC                | Model, View, Controller                            | Small apps, rapid prototypes      |
| MVP                | Passive view; Presenter decides all                | High unit-test coverage           |
| MVVM               | Reactive binding; View Model has no View reference | Modern Android, Flutter, SwiftUI  |
| Clean Architecture | Dependencies point inward only                     | Large, long-lived enterprise apps |
| BLoC               | Unidirectional: Events in, States out              | Complex shared state in Flutter   |
| Redux              | Single global immutable store                      | Large React Native apps           |

*Table 1.3 Architectural patterns summarized by principle and context.*

For CarDiag, a layered structure was adopted on both ends of the system. On the backend, described fully in Chapter 6, the Spring Boot server follows a Controller–Service–Repository–Entity layering that mirrors the separation-of-concerns principle common to MVC and Clean Architecture. On the frontend, the React Native codebase is organized by feature (screens, components, services, store) as detailed in Chapter 5, keeping presentation, navigation, and business logic in clearly separated folders so that any single screen can be modified without destabilizing the rest of the application.

## 1.5 Requirement Engineering Process

Requirement engineering is the structured process of discovering, analyzing, documenting, and validating what a system must do before implementation begins. On mobile projects, inadequate requirements are among the most common causes of cost overrun and rework, which is why the process was treated as a dedicated phase rather than an informal precursor to design. It begins with stakeholder identification, since different groups contribute different information and terminology, followed by elicitation, analysis, classification, prioritization, and validation with stakeholders.

| Technique             | Purpose                                                 |
|-----------------------|---------------------------------------------------------|
| Structured interviews | Understand decision-making processes and business rules |

|                                  |                                                                             |
|----------------------------------|-----------------------------------------------------------------------------|
| Surveys and questionnaires       | Identify frequency-of-use patterns and priority rankings at scale           |
| Observation / contextual inquiry | Reveal unstated assumptions and workarounds                                 |
| Workshops and brainstorming      | Generate requirements collaboratively; surface conflicting priorities early |
| Reverse engineering              | Derive a baseline of functionality from existing or competing applications  |
| Prototyping                      | Convert abstract requirements into concrete discussion using mockups        |

*Table 1.6 Requirement elicitation techniques reviewed for this project.*

Chapters 2 and 3 of this report present the full application of this process to CarDiag, including which of these techniques were used and what each contributed to the final requirement set.

## 1.6 Mobile App Development Cost Estimation

Cost estimation for a mobile application begins with a well-defined scope target platforms, development approach, feature set, and backend dependencies and proceeds by decomposing features into complexity bands, applying representative hourly rates, and adding overhead for project management, QA, UI/UX design, and contingency. Table 1.4 summarizes the complexity bands used as a reference framework.

| Complexity band | Estimated hours | Representative features                        |
|-----------------|-----------------|------------------------------------------------|
| Simple          | 20 – 40 hours   | Static screens, basic navigation, login/logout |
| Medium          | 40 – 80 hours   | User profiles, push notifications, search      |
| Complex         | 80+ hours       | Real-time messaging, offline sync, ML features |

*Table 1.4 Feature complexity bands and indicative hour ranges.*

Applying this framework, CarDiag's feature set spans all three bands: authentication and profile screens are simple; history, tutorials, and navigation are medium; and the AI-driven image, audio, and vector-similarity diagnosis pipeline is complex. Hourly rates vary with developer



seniority and geographic location, both of which must be factored into a realistic budget, as summarized in Table 1.5.

| Seniority / Region                      | Hourly rate (USD) |
|-----------------------------------------|-------------------|
| Junior developer                        | \$25 – \$50       |
| Mid-level developer                     | \$50 – \$100      |
| Senior developer                        | \$100 – \$150+    |
| Asia (region)                           | \$15 – \$40       |
| Latin America (region)                  | \$40 – \$75       |
| Western Europe / North America (region) | \$80 – \$150+     |

*Table 1.5 Indicative hourly rates by developer seniority and region.*

The realistic total cost of a project of this scope is calculated as the base development cost (estimated hours × hourly rate) multiplied by a combined overhead and contingency factor of 1.40 to 1.50, to account for project management (10–15%), testing and QA (10–15%), UI/UX design (10–20%), and a contingency buffer (10–20%) for scope changes and integration surprises a discipline the team applied qualitatively when scoping the seven tasks of this project against the semester timeline.

## 1.7 Chapter Summary

This chapter has surveyed the foundational technology decisions that precede any mobile software project and stated the choices made for CarDiag: a cross-platform React Native frontend, a layered Spring Boot backend, and a structured requirement engineering process to be documented in the chapters that follow. The remaining six chapters trace how these decisions were carried through requirement gathering, requirement analysis, system modelling, interface design, database implementation, and final delivery.

## 2. REQUIREMENT GATHERING

### 2.1 Introduction

Requirement gathering is a foundational activity in the software engineering lifecycle, and for an AI-powered diagnostic application it is especially critical that the resulting system align with real user needs, technical feasibility, and operational constraints. This chapter documents the stakeholders identified for CarDiag, the techniques used to elicit requirements, the data gathered for the underlying AI components, the cleaning applied to that data, and an assessment of anticipated user reluctance toward the proposed system.

### 2.2 Stakeholder Identification

Stakeholders were grouped into three tiers. Primary stakeholders interact directly with the system: car owners and everyday drivers who need to interpret warning lights and engine sounds without a mechanic, the development team who need clear implementable requirements, and UX/UI designers who need to understand user demographics and literacy levels. Secondary stakeholders influence development without using the system directly, including automobile manufacturers who provide OBD-II symbol definitions and YouTube content creators who supply repair tutorial content. Tertiary stakeholders, such as road safety and environmental authorities, are affected indirectly through the broader effects of fewer undetected vehicle faults.

| Stakeholder                               | Tier      | Primary interest                                                |
|-------------------------------------------|-----------|-----------------------------------------------------------------|
| Car owners / drivers                      | Primary   | Interpret warning lights and sounds without a mechanic          |
| Mobile app developers                     | Primary   | Clear, implementable functional and non-functional requirements |
| UX/UI designers                           | Primary   | User demographics, literacy levels, interaction preferences     |
| Automobile manufacturers (OEMs)           | Secondary | Standardized OBD-II symbol definitions                          |
| YouTube content creators / repair experts | Secondary | Providers of tutorial video content                             |

|                                           |          |                                                       |
|-------------------------------------------|----------|-------------------------------------------------------|
| Road safety and environmental authorities | Tertiary | Fewer vehicle failures and undetected emission faults |
|-------------------------------------------|----------|-------------------------------------------------------|

*Table 2.1 Stakeholders identified for the CarDiag project.*

## 2.3 Requirement Gathering Techniques

No single elicitation technique is sufficient on its own, so five complementary techniques were combined to reduce the risk of missing critical requirements.

- Surveys: a structured questionnaire (online via Google Forms, and on paper) was completed by 80 respondents drawn from university staff, students, and the wider Buea community, covering warning-light familiarity, willingness to use audio-based diagnosis, and offline requirements. Approximately 65% of respondents reported encountering a warning light they could not interpret.
- Interviews: semi-structured interviews of 30–60 minutes were held with five mechanics and three experienced car owners, surfacing insights such as drivers routinely ignoring the check-engine light and the same warning symbol carrying different severity depending on vehicle make and year.
- Brainstorming: a facilitated team session produced the five core feature modules (Camera Scanner, Audio Recorder, Fault Explainer, Urgency Estimator, Video Tutorial Linker), confirmed the need for offline-first architecture, and settled on React Native with a hybrid on-device / cloud fallback diagnosis strategy.
- Reverse engineering: three existing diagnostic apps (Car Scanner ELM OBD2, FIXD, AutoDoc) were installed and analyzed. All three required a Bluetooth OBD-II dongle and none supported audio-based fault detection confirming a clear differentiator for CarDiag.
- Document analysis: the OBD-II standard (SAE J1979), ISO 11898 (CAN bus), and published audio datasets (ESC, ARCA) were reviewed to ground the fault and sound taxonomies used later in the database design.

## 2.4 Data Gathering and Cleaning

Two categories of data were gathered for the AI components: dashboard warning-light images, sourced from ISO 2575 symbol libraries, web scraping, and primary photography of ten vehicle models; and engine audio recordings, sourced from the AudioSet and ESC-50 public datasets

together with primary recordings of six fault types made with a mechanic partner. Survey and interview data were also captured as structured records. Systematic cleaning was then applied: duplicate removal via perceptual hashing, resolution filtering, annotation correction, silence trimming, signal-to-noise filtering, resampling to a uniform rate, and label verification against the mechanic partner's descriptions.

| Dataset          | Raw count | Cleaned count | Reduction |
|------------------|-----------|---------------|-----------|
| Dashboard images | 1,240     | 1,030         | 16.9%     |
| Audio clips      | 3,600     | 3,299         | 8.4%      |
| Survey responses | 80        | 69            | 13.8%     |

*Table 2.2 Summary of raw versus cleaned dataset volumes.*

Table 2.5 breaks down the sources used to build the two primary AI training datasets before cleaning was applied.

| Data category            | Volume collected     | Format             | Source type                         |
|--------------------------|----------------------|--------------------|-------------------------------------|
| OBD-II standard symbols  | 87 unique symbols    | SVG / PNG          | ISO standard library                |
| Real dashboard photos    | 1,240 images         | JPEG               | Web scraping + primary photography  |
| Annotated bounding boxes | 1,240 entries        | JSON (COCO)        | Manual annotation                   |
| Engine audio recordings  | 3,600 clips (10–30s) | WAV, 44.1 kHz mono | Public datasets + primary recording |
| Fault–audio label pairs  | 3,600 entries        | CSV                | Expert annotation                   |

*Table 2.5 Data sources used to build the image and audio training datasets.*

## 2.5 User Reluctance Assessment

Five sources of potential user reluctance were identified from the survey and interview data, each mapped to a concrete design or policy mitigation built into the requirements: privacy concerns over camera and microphone access (41% of respondents), distrust of AI diagnostic accuracy (raised by four of five mechanics interviewed), digital literacy barriers (28% overall, rising to 54% among respondents over 45), cost and storage concerns (18–23%), and a safety over/under-confidence paradox in how urgency should be communicated.

| Reluctance factor            | Prevalence      | Planned mitigation                                     |
|------------------------------|-----------------|--------------------------------------------------------|
| Privacy / data concerns      | 41%             | On-device processing; explicit permission prompts      |
| Distrust of AI accuracy      | ~67% of experts | Confidence scores; professional referral prompts       |
| Digital literacy barriers    | 28–54%          | Simplified UI; bilingual support; voice guidance       |
| Cost / storage concerns      | 18–23%          | Free tier; modular offline models; <80 MB base install |
| Safety over/under-confidence | Qualitative     | Conservative urgency calibration                       |

*Table 2.3 Reluctance factors and their planned mitigations.*

## 2.6 Consolidated High-Level Requirements

The stakeholder analysis, elicitation activities, and reluctance assessment together produced an initial set of high-level requirement categories, summarized in Table 2.4, which were carried forward into Chapter 3 for formal analysis, classification, and prioritization.

| ID    | High-level requirement                                                             |
|-------|------------------------------------------------------------------------------------|
| FR-01 | Capture and classify dashboard warning light symbols using the device camera       |
| FR-02 | Detect and classify engine fault types from audio recordings                       |
| FR-03 | Support simultaneous detection of multiple active warning lights                   |
| FR-04 | Provide explanations, probable causes, and urgency ratings for each detected fault |
| FR-05 | Link users to relevant tutorial videos from verified automotive professionals      |
| FR-06 | Operate core diagnostic features offline using pre-downloaded models               |
| FR-07 | Synchronize with online fault databases when connectivity is available             |

|        |                                                                        |
|--------|------------------------------------------------------------------------|
| NFR-01 | Complete visual fault classification within 3 seconds of image capture |
| NFR-02 | Achieve a minimum of 85% accuracy on the validation dataset            |
| NFR-03 | Keep the offline base install under 80 MB                              |
| NFR-04 | Support Android 9.0+ and iOS 13.0+                                     |
| NFR-05 | Comply with data-minimization and purpose-limitation principles        |

*Table 2.4 Consolidated high-level functional and non-functional requirements.*

## 3. REQUIREMENT ANALYSIS

### 3.1 Introduction

Following requirement gathering, the raw requirements were analyzed for completeness, clarity, technical feasibility, and internal consistency, then classified, prioritized, and consolidated into a Software Requirements Specification (SRS). This chapter presents the results of that analysis, drawing on the 80-respondent survey introduced in Chapter 2.

### 3.2 Completeness, Clarity, and Feasibility

The requirement set was found to cover the core features needed for the system: OBD-II based fault detection, image- and sound-based AI diagnosis, bilingual support, maintenance history, nearby-mechanic suggestions, cost estimation, and the associated non-functional requirements for performance, offline mode, and data security. Review nonetheless surfaced four completeness gaps: no defined behavior for unrecognized faults, no handling of noisy audio environments, no specification for connectivity loss mid-diagnosis, and no guidance for low-end device performance.

Several requirements were rewritten to be measurable. "The app should detect engine sounds" became a requirement to identify at least five common abnormal engine sounds at a minimum 85% accuracy; "fast image processing" became a 10-second maximum diagnosis time under normal network conditions; and "support multiple languages" was clarified to mean English and French, with future support for Cameroonian Pidgin English. On feasibility, OBD-II integration was confirmed as well-established for vehicles manufactured after 1996, while full offline AI diagnosis was judged unrealistic given server-side model requirements leading to the hybrid offline/online approach carried through the rest of the project.

### 3.3 Inconsistencies, Ambiguities, and Missing Information

| Issue                           | Description                                                                | Resolution                                                              |
|---------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Offline vs online functionality | Some requirements assume constant internet; others require offline support | Tiered approach: core features offline, advanced AI requires connection |

|                                  |                                                          |                                                                      |
|----------------------------------|----------------------------------------------------------|----------------------------------------------------------------------|
| AI accuracy across devices       | Consistent performance expected despite varying hardware | Set minimum hardware requirements; test on low- and high-end devices |
| Safety while driving             | Real-time audio monitoring may distract the driver       | Limit active diagnosis to when the vehicle is stationary             |
| Technical vs non-technical users | Mechanics want detail; drivers want simplicity           | Simple mode and expert mode, user-selectable                         |

*Table 3.1 Inconsistencies identified during requirement analysis and their resolutions.*

Ambiguities were also resolved: cost estimation will present price ranges rather than exact figures, given that prices vary by mechanic; "nearby" mechanic suggestions were bounded by a default, user-adjustable search radius. Missing requirements were added to cover unknown-fault handling, data retention and deletion policy, and remote updates to the fault database without requiring a full app release.

### 3.4 Requirement Prioritization (MoSCoW)

Requirements were prioritized using the MoSCoW method to focus the initial release on what matters most.

| Priority        | Representative features                                                                                                                              |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Must have       | Fault detection (OBD-II / AI image / audio); plain-language explanations; step-by-step guidance; bilingual support; authentication and data security |
| Should have     | Maintenance history and reminders; nearby mechanic suggestions; cost estimation; offline mode for basic features                                     |
| Could have      | Predictive maintenance alerts; fuel efficiency tips; community forum; voice assistant integration                                                    |
| Won't have (v1) | Real-time mechanic chat; insurance claim assistance; full offline AI diagnosis                                                                       |

*Table 3.2 Requirement prioritization using the MoSCoW method.*

### 3.5 Requirement Classification



### 3.5.1 Functional Requirements

- Connect to a vehicle's OBD-II port via Bluetooth or Wi-Fi and read fault codes.
- Capture a photo of a dashboard warning light or component and return a diagnosis.
- Detect and classify abnormal engine sounds from the device microphone.
- Explain detected faults and warning lights in plain, non-technical language.
- Suggest nearby mechanics based on the user's GPS location.
- Store and display vehicle maintenance history, with service reminders.
- Estimate repair costs based on the fault detected and the user's location.
- Support English and French, selectable during onboarding.

### 3.5.2 Non-Functional Requirements

| Category        | Requirement                                                                        |
|-----------------|------------------------------------------------------------------------------------|
| Performance     | Diagnosis from an image within 10 seconds under a standard 4G connection           |
| Reliability     | System available at least 99% of the time during normal operating hours            |
| Usability       | First-time user completes a basic fault check within 3 minutes, unguided           |
| Security        | All user data encrypted in transit and at rest; optional two-factor authentication |
| Compatibility   | Android 8.0+ and iOS 13+; standard OBD-II Bluetooth and Wi-Fi adapters             |
| Offline support | Warning-light lookup and basic fault explanations accessible without internet      |
| Localization    | Fault database includes vehicles common in Cameroon (Toyota, Nissan, Mitsubishi)   |

*Table 3.3 non-functional requirements by category.*

## 3.6 Software Requirements Specification (SRS) Summary

The system purpose defined in the SRS is a mobile application that enables drivers, car owners, and mechanics in Cameroon to detect, understand, and respond to vehicle faults using AI-

powered tools, functioning as a decision-support and information tool rather than a replacement for professional assessment. Its scope covers fault detection, maintenance tracking, mechanic discovery, and cost awareness, explicitly excluding vehicle repair, mechanic booking, and insurance processing. Five system interfaces were specified OBD-II, camera, microphone, GPS, and cloud storage together with error-handling behavior for OBD-II connection failure, unrecognized faults, connectivity loss mid-session, and unavailable location services.

### 3.7 System Interfaces and Error Handling

The SRS defines five system interfaces through which CarDiag exchanges data with its environment, and a corresponding error-handling behavior for each, so that failures degrade gracefully rather than losing user data or crashing the application.

| Interface                  | Purpose                                               | Failure behavior                                                          |
|----------------------------|-------------------------------------------------------|---------------------------------------------------------------------------|
| OBD-II (Bluetooth / Wi-Fi) | Reads diagnostic trouble codes from the vehicle ECU   | Notify the user and offer image- or sound-based diagnosis instead         |
| Camera                     | Captures dashboard warning light images               | If no fault is recognized, inform the user and recommend a mechanic visit |
| Microphone                 | Captures engine and mechanical sounds                 | Prompt the user to re-record in a quieter environment                     |
| GPS                        | Locates nearby mechanics and localizes cost estimates | Continue core diagnosis without the mechanic-suggestion feature           |
| Cloud storage              | Stores maintenance records and user data securely     | Save the session locally and resume sync once connectivity returns        |

*Table 3.4 System interfaces and their defined error-handling behavior.*

### 3.8 Stakeholder Validation

The consolidated requirements were validated against the three main stakeholder groups. Drivers confirmed the need for simple language, offline access, and affordability. Mechanics confirmed the importance of OBD-II integration, accurate local fault codes, and an expert display mode, while noting that generic international fault databases often miss patterns specific

to older vehicles common in Cameroon. Fleet managers confirmed the value of multi-vehicle maintenance tracking, service reminders, and cost visibility.

Table 3.5 summarizes how each stakeholder group's concerns were traced to specific validated requirements.

| Stakeholder group | Key concerns                                                   | Confirmed requirements                                        |
|-------------------|----------------------------------------------------------------|---------------------------------------------------------------|
| Drivers           | Simple language, affordable access, offline mode               | Fault explanation, step-by-step guidance, bilingual support   |
| Mechanics         | Accurate fault codes, local vehicle database, technical detail | OBD-II integration, local fault database, expert display mode |
| Fleet managers    | Multi-vehicle tracking, maintenance records, cost visibility   | Maintenance history, service reminders, cost estimation       |

*Table 3.5 Stakeholder validation summary.*

The requirement analysis confirmed a genuine need for an accessible, bilingual, AI-powered diagnostic tool designed around the Cameroonian context, with fault detection, warning-light explanation, step-by-step guidance, and basic security identified as the features to build first. The main risks carried forward into system design were offline functionality, AI accuracy on low-end devices, and the completeness of the local vehicle fault database.

## 4. SYSTEM MODELLING AND DESIGN

### 4.1 Introduction

This chapter presents the system diagrams produced for CarDiag: the context diagram, the data flow diagram, the use case diagram, representative sequence diagrams, the class diagram, and the deployment diagram. Together they define the structural and behavioral blueprint that guided implementation.

### 4.2 Context Diagram

The context diagram treats the entire application as a single process and shows the actors it exchanges data with. Six external actors were identified: the driver (the primary user, exchanging photos, audio, GPS location, and login credentials for fault explanations and guidance), the vehicle's OBD-II port (a one-way source of diagnostic trouble codes), the mechanic and fleet manager (secondary actors whose flows all require internet connectivity), the fault code library, and a remote server for heavier processing.

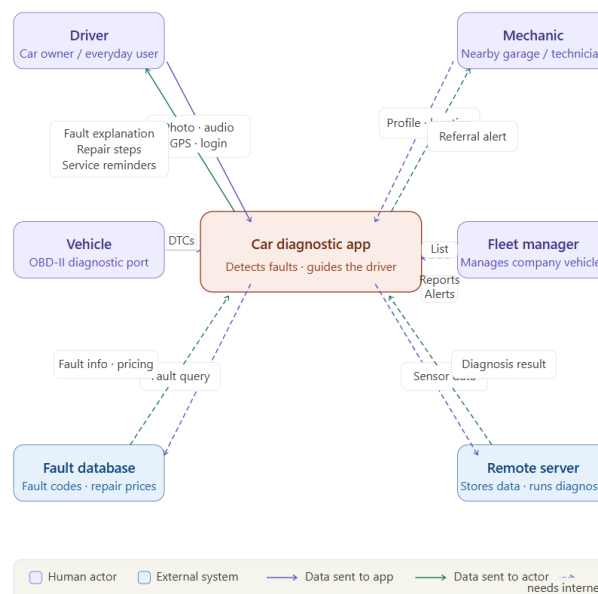


Figure 4.1 Context diagram of the CarDiag system.

The driver is the most central actor, confirming that the interface must prioritize a simple, non-technical experience. The vehicle communicates one-way only, simplifying OBD-II integration since the app never writes to the vehicle ECU. Two tiers of external system a cached fault library and a remote server directly support the hybrid offline/online architecture established during requirement analysis.

### 4.3 Data Flow Diagram

The Level 1 data flow diagram expands the context diagram into six internal processes: verify user identity, capture vehicle data, analyze the fault, build diagnosis report, record in history, and find nearby mechanic supported by five data stores: user accounts, the fault code library, maintenance history, the mechanic directory, and a repair price list.

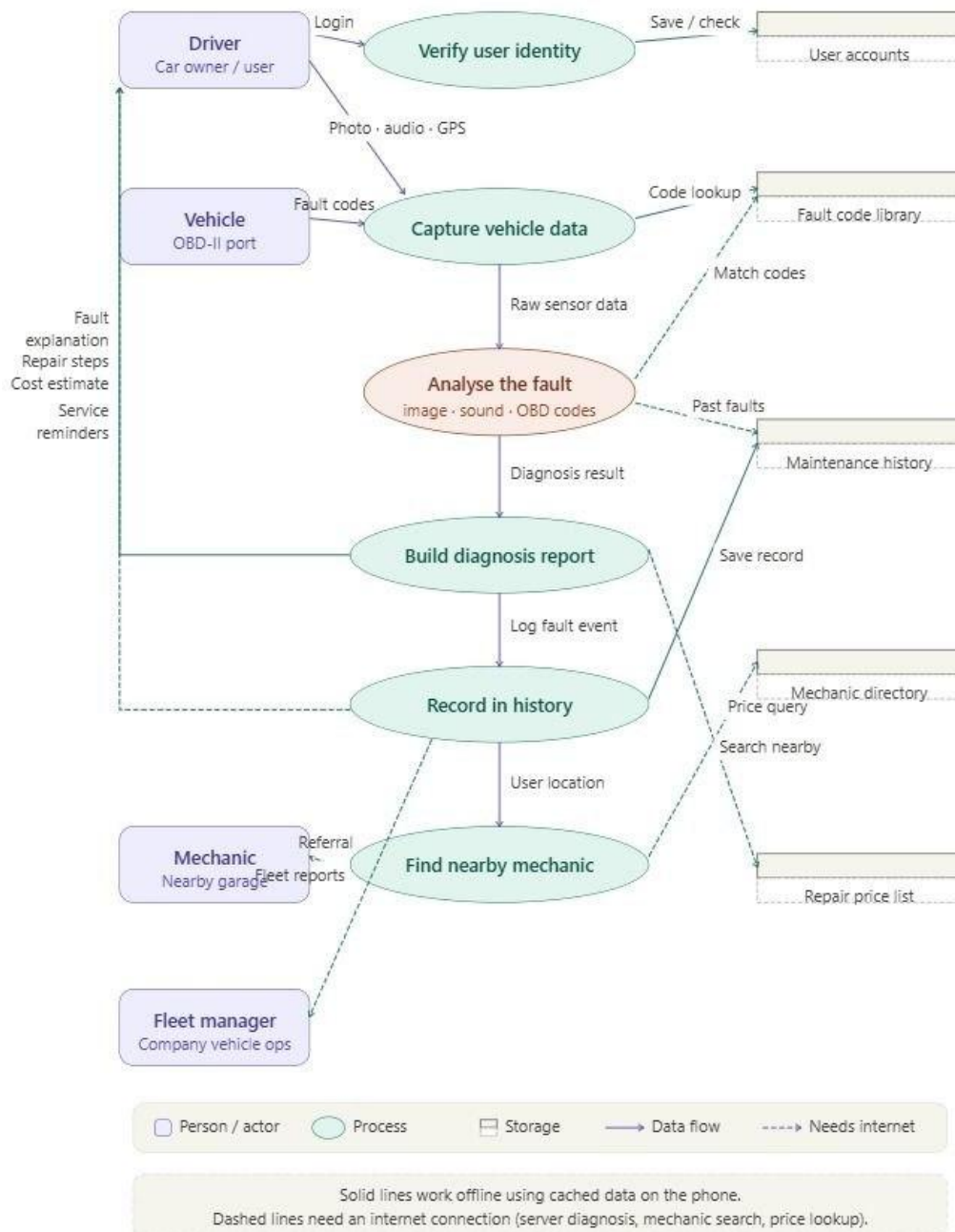


Figure 4.2 Level 1 data flow diagram of the CarDiag system.

Five data stores support the six processes, represented in the diagram as open-ended rectangles, the standard DFD notation for a data store.

| Storage             | Contents and usage                                                                              |
|---------------------|-------------------------------------------------------------------------------------------------|
| User accounts       | Driver profiles, login credentials, vehicle registration; read/written by identity verification |
| Fault code library  | Catalogue of known fault codes with descriptions and repair cost ranges; updated remotely       |
| Maintenance history | Past fault events per vehicle; supports predictive maintenance alerts                           |
| Mechanic directory  | Registered mechanics with location and contact information; requires internet                   |
| Repair price list   | Estimated repair costs per fault type and geographic area                                       |

*Table 4.1 Data stores supporting the six internal processes of the DFD.*

The fault analysis process is the most connected node, reading from two stores and receiving data from the capture process, confirming it as the most complex process to implement and the one most likely to require performance optimization on low-end phones. Flows to the mechanic locator and fleet manager are entirely dashed (internet-dependent), consistent with their status as secondary, deferrable features.

#### 4.4 Use Case Diagram

The use case model defines two primary actors the Car Owner and the Admin and four secondary actors: the AI Recognition Engine, the AI Audio Engine, the Local AI Engine, and the Video Platform. Nine use cases are enclosed within the system boundary, covering registration and login, dashboard warning-light scanning, engine sound recording, consolidated diagnosis results, repair tutorials, diagnosis history, offline diagnosis, vehicle profile management, and fault database updates.

## AI-Based Car Fault Diagnosis System

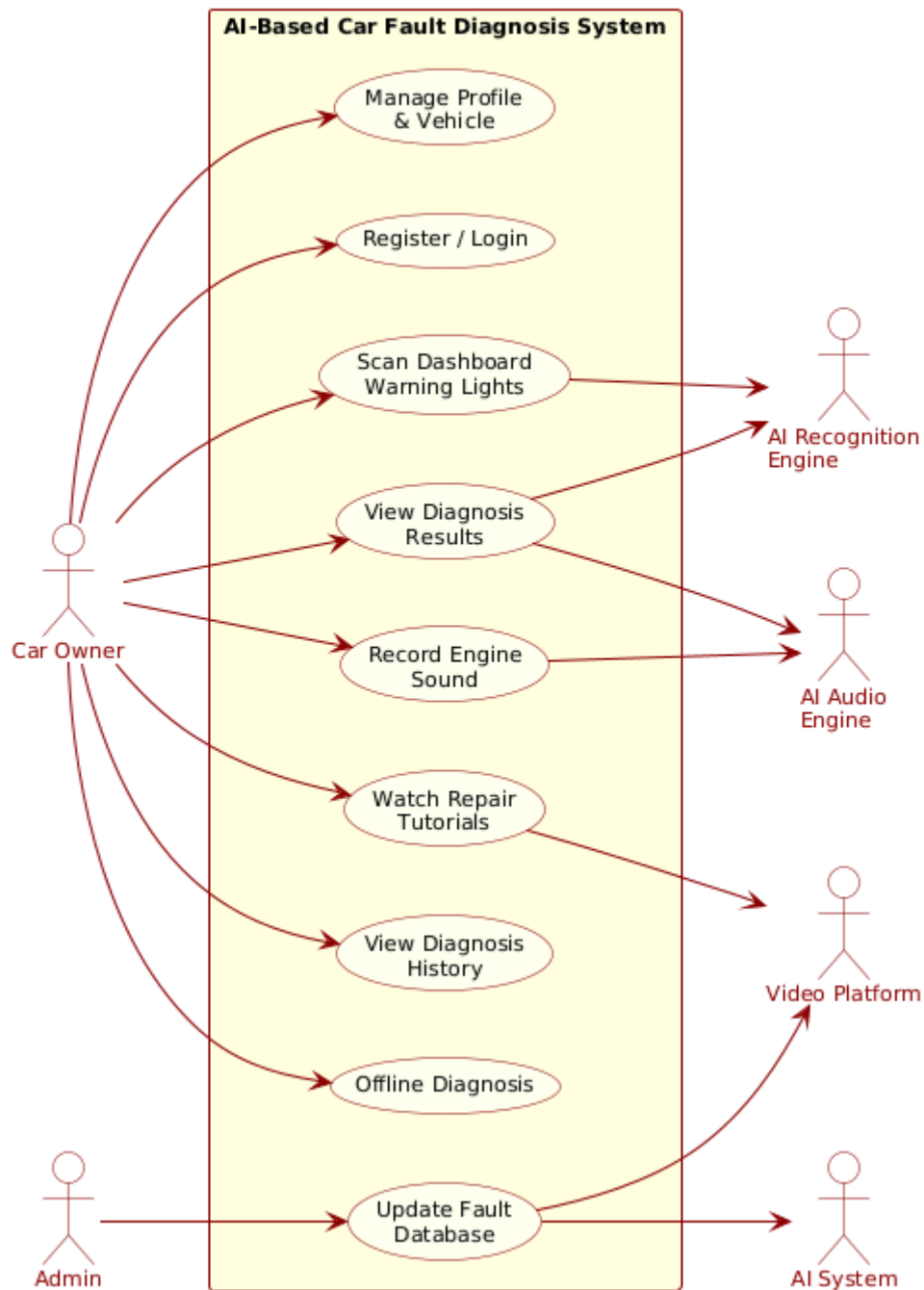


Figure 4.3 Main system use case diagram for CarDiag.

| Use case                      | Primary actor | Summary                                                     |
|-------------------------------|---------------|-------------------------------------------------------------|
| Register / Login              | Car Owner     | Create or access an account to sync history and preferences |
| Scan dashboard warning lights | Car Owner     | Capture and classify illuminated dashboard symbols          |

|                                 |           |                                                        |
|---------------------------------|-----------|--------------------------------------------------------|
| Record and analyze engine sound | Car Owner | Capture and classify abnormal engine audio             |
| View diagnosis results          | Car Owner | Consolidate image and audio findings into one report   |
| Watch repair tutorials          | Car Owner | Fetch and stream matched YouTube tutorials             |
| Perform offline diagnosis       | Car Owner | Fall back to the on-device model when disconnected     |
| Manage profile and vehicle      | Car Owner | Maintain vehicle details and browse diagnosis history  |
| Update fault database           | Admin     | Push new fault signatures and model weights to devices |

*Table 4.2 Summary of the principal CarDiag use cases.*

Table 4.3 lists the actors that appear across the full use case model, including the secondary AI and platform actors that support the Car Owner's primary interactions.

| Actor                 | Type      | Description                                                       |
|-----------------------|-----------|-------------------------------------------------------------------|
| Car Owner             | Primary   | Main user who interacts with the mobile application               |
| Admin                 | Primary   | Maintains system datasets and updates AI models                   |
| AI Recognition Engine | Secondary | External AI component responsible for dashboard image recognition |
| AI Audio Engine       | Secondary | External AI component responsible for engine sound analysis       |
| Local AI Engine       | Secondary | Offline on-device component used without internet connectivity    |
| Video Platform        | Secondary | External system providing repair and maintenance tutorial videos  |

*Table 4.3 Actors identified in the CarDiag use case model.*



## 4.5 Sequence Diagrams

Eight sequence diagrams were developed to describe the dynamic behavior behind each major use case, covering registration/login, dashboard scanning, engine sound analysis, consolidated results, tutorial retrieval, and the offline fallback path. Figure 4.4 reproduces the dashboard-scanning sequence as a representative example: the mobile app forwards the captured image to the AI Recognition Engine, which normalizes the frame, runs object detection, and classifies detected symbols; matched identifiers are then queried against the fault database, which returns the meaning, probable cause, and urgency level, or prompts the user to adjust the camera if no symbol is detected.

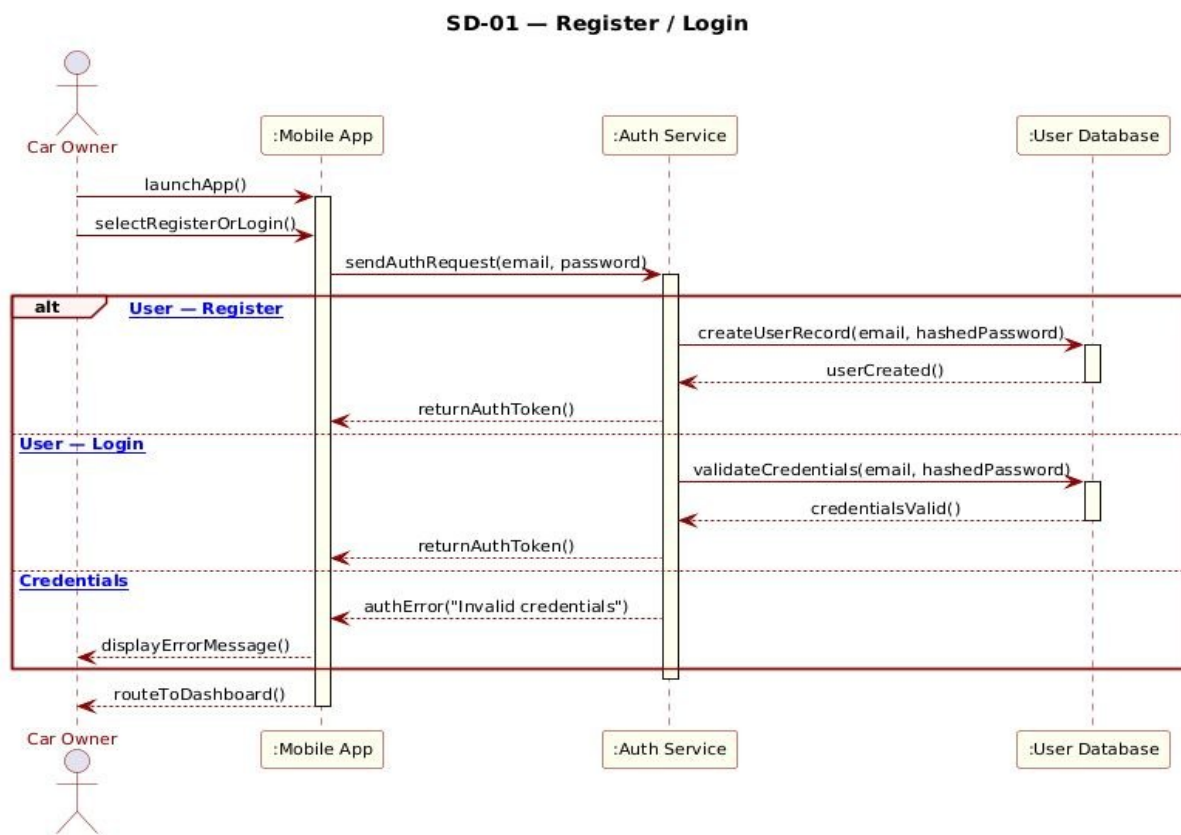


Figure 4.4 Sequence diagram for the dashboard warning-light scanning use case.

Figure 4.5 reproduces a second representative sequence the offline diagnosis fallback which is triggered whenever the mobile app detects no network connection. Processing is rerouted away from the cloud AI services to the on-device Local AI Engine, which queries the local fault database directly and returns a basic diagnosis report, while the user is notified that richer video content will remain unavailable until connectivity is restored.

## SD-06 — Offline Diagnosis

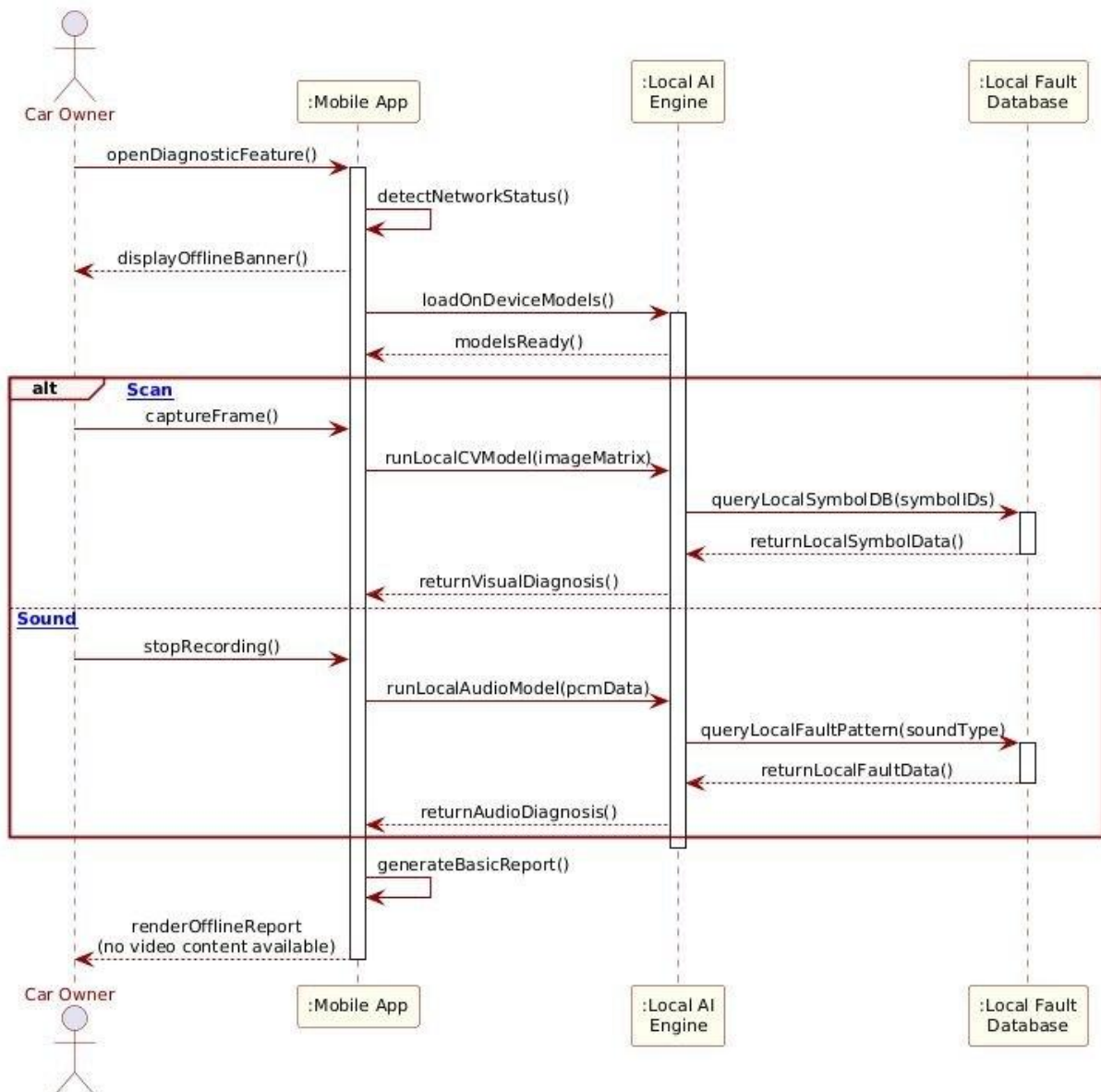


Figure 4.5 Sequence diagram for the offline diagnosis fallback.

## 4.6 Class Diagram

The class diagram models the static structure of CarDiag's core domain around five classes. Fault sits at the center, holding a fault's name, category, urgency level, causes, and repair tips. FaultEmbedding and Video are connected to Fault by one-to-one and one-to-many composition respectively, meaning neither can exist independently of the fault they describe. User connects to Diagnosis through a one-to-many aggregation, allowing diagnoses to be created by guests and to persist independently of any account, while Diagnosis is associated with Fault through an "is matched to" relationship.

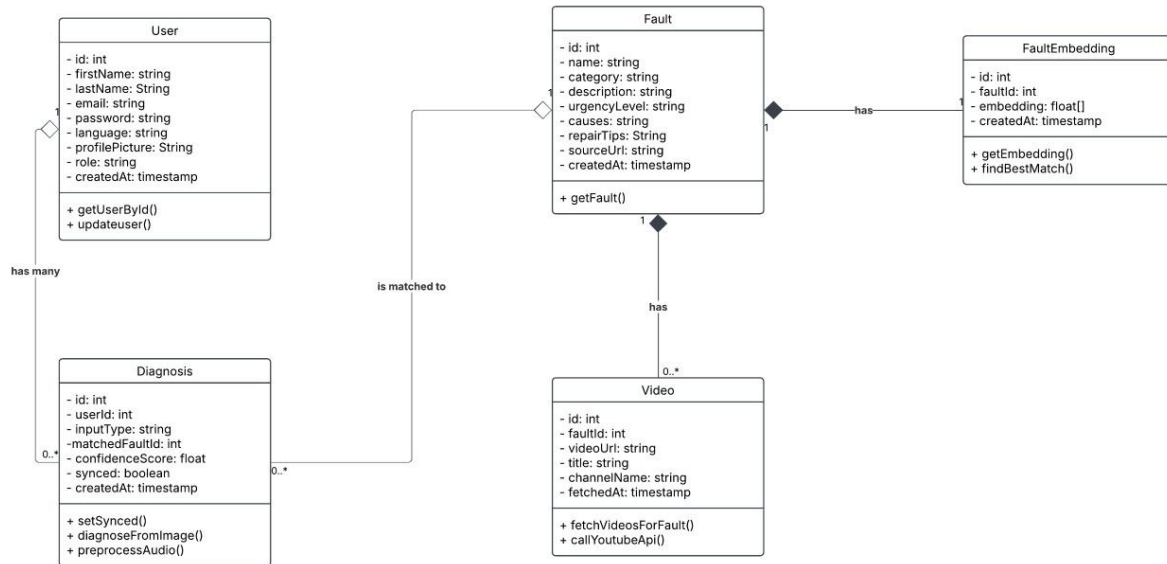


Figure 4.6 Class diagram of the CarDiag domain model.

This design cleanly separates data ownership (composition, for embeddings and videos strictly tied to a fault) from data reference (aggregation and association, for users and diagnoses), directly supporting the system's offline-first strategy and its requirement to maintain a complete diagnosis history per user, including guests.

## 4.7 Deployment Diagram

The deployment diagram shows how CarDiag is physically distributed across a mobile device, a backend server, a database server, and two external services. The mobile device hosts the application itself, an on-device TensorFlow Lite model, and a local SQLite cache, enabling offline fault lookup and local inference. It communicates with the backend server over HTTPS, which in turn queries the database server over SQL; it also communicates directly with the YouTube API over HTTPS for tutorial content, and optionally with an OBD-II adapter over Bluetooth.

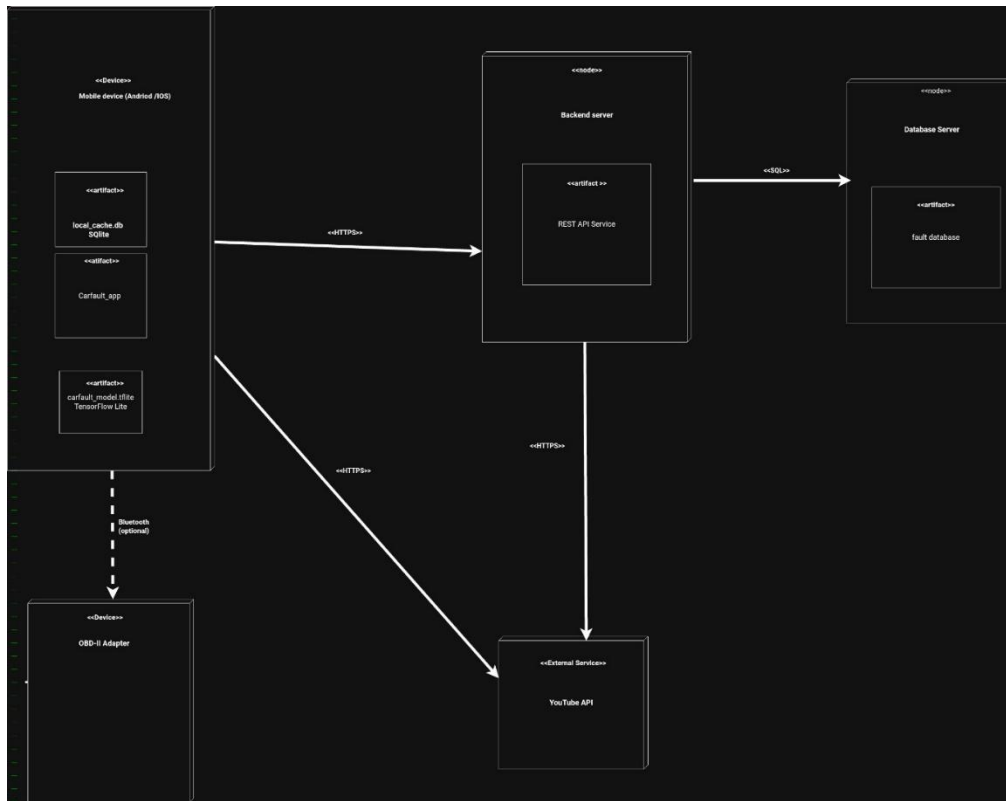


Figure 4.7 Deployment diagram of the CarDiag system.

This hybrid deployment gives CarDiag offline capability through its local artifacts, fast on-device inference through the TensorFlow Lite model, and cloud synchronization through the backend and database servers, while treating OBD-II hardware integration as an optional, not mandatory, diagnostic channel.

## 5. UI DESIGN AND IMPLEMENTATION

### 5.1 Introduction

With the system modelled, the next task was to define CarDiag's identity and translate its requirements into a concrete, usable interface. This chapter covers the application's technical and visual identity, the design objectives and principles that guided the interface, the color palette and typography, the ten core screens, and the React Native frontend implementation.

### 5.2 Application Identity

CarDiag is registered under the package name `com.group12.cardiag` on both Android and iOS, targeting Android 8.0 (API 26) and iOS 8 as minimum versions, with Android 13 and iOS 14 as target versions. The application follows Semantic Versioning, beginning at 1.0.0. It requires internet, Bluetooth, camera, and microphone permissions to support cloud synchronization, OBD-II adapter connectivity, dashboard scanning, and engine sound recording respectively.



*Figure 5.1 The CarDiag logo, combining a tachymeter instrument with a vehicle silhouette.*

The logo combines a circular tachymeter with an embedded car silhouette: the arc transitions from blue to orange to represent the shift from normal operation to a fault state, with the needle pointing into the fault zone. The wordmark sets "Car" and "Diag" in contrasting weights above the tagline "Smart Car Fault Diagnosis". The app icon (Figure 5.2) is a square, text-free reduction of the same symbol, sized for home-screen and store-listing use across both platforms.



*Figure 5.2 The CarDiag app icon.*

### 5.3 Design Objectives and Principles

The visual design was guided by six objectives: a modern, professional interface that communicates automotive diagnostics clearly; direct access to key features without deep navigation; readable diagnostic information on small screens; visual consistency through a shared component library; equal support for Android and iOS; and immediate visual feedback after actions such as scanning, recording, and diagnosing.

- Simplicity each screen is built around a single task, with every major feature one tap away via the bottom navigation bar.
- Consistency buttons, inputs, cards, and status badges share the same visual style across all screens.
- Accessibility text size, color contrast, and touch target size were chosen for readability by all users.
- Feedback scanning shows a live viewfinder, recording shows a live waveform and timer, and diagnosis results use color-coded severity badges.

### 5.4 Color Palette and Typography

| Color                   | Hex code | Usage                                         |
|-------------------------|----------|-----------------------------------------------|
| Electric Blue (primary) | #1E88E5  | Buttons, links, active states, brand elements |

|                         |         |                                             |
|-------------------------|---------|---------------------------------------------|
| Deep Navy (background)  | #0A1628 | App background, icon background             |
| Carbon Black            | #070E1C | Card backgrounds, secondary surfaces        |
| Warning Orange (accent) | #FF6D00 | Fault alerts, warning indicators, needle    |
| Success Green           | #4CAF50 | Normal vehicle status, completed operations |
| Error Red               | #D32F2F | Critical fault states only                  |

*Table 5.1 CarDiag color palette and usage (rendered in greyscale for print).*

Electric Blue was chosen to convey technology and reliability, Deep Navy gives the interface a professional, high-contrast look inspired by real OBD diagnostic tools, and Warning Orange is reserved exclusively for fault alerts, creating an immediate visual association between color and vehicle fault. Three typefaces cover all text roles: Inter Bold for headings and screen titles, Roboto Regular for body text and inputs, and Roboto Mono for fault codes and version badges, whose fixed-width characters make alphanumeric codes easier to read at a glance.

## 5.5 Navigation and Components

CarDiag uses a five-tab bottom navigation bar, placing every major feature one tap away from any screen, as summarized in Table 5.3.

| Tab       | Function                                     |
|-----------|----------------------------------------------|
| Home      | Dashboard and vehicle status                 |
| Scan      | Warning light scanner and engine sound check |
| History   | Diagnosis records                            |
| Tutorials | Repair guides                                |
| Profile   | User account and settings                    |

*Table 5.3 The five tabs of the CarDiag bottom navigation bar.*

The component library defines primary buttons (solid Electric Blue), secondary buttons (outlined), and destructive buttons (solid red) for actions such as Stop Recording; cards with a Deep Navy background and subtle border for grouped information; input fields with contextual

icons and inline validation; and status badges using orange, red, and blue to indicate medium, high, and normal severity, as summarized in Table 5.4.

| Severity badge | Color  | Meaning                                             |
|----------------|--------|-----------------------------------------------------|
| Normal         | Blue   | No critical faults detected                         |
| Medium         | Orange | Fault present; monitor and schedule repair          |
| High           | Red    | Urgent fault; seek professional inspection promptly |

*Table 5.4 Severity badge color coding used across diagnosis and history screens.*

## 5.6 Screen Design

Ten core screens were designed and carried through to implementation: Splash, Login, Register, Dashboard, Warning Light Scanner, Engine Sound Recording, Diagnosis Results, Repair Tutorials, History, and Profile. Figure 5.3 shows the high-fidelity mockups for all ten screens together.



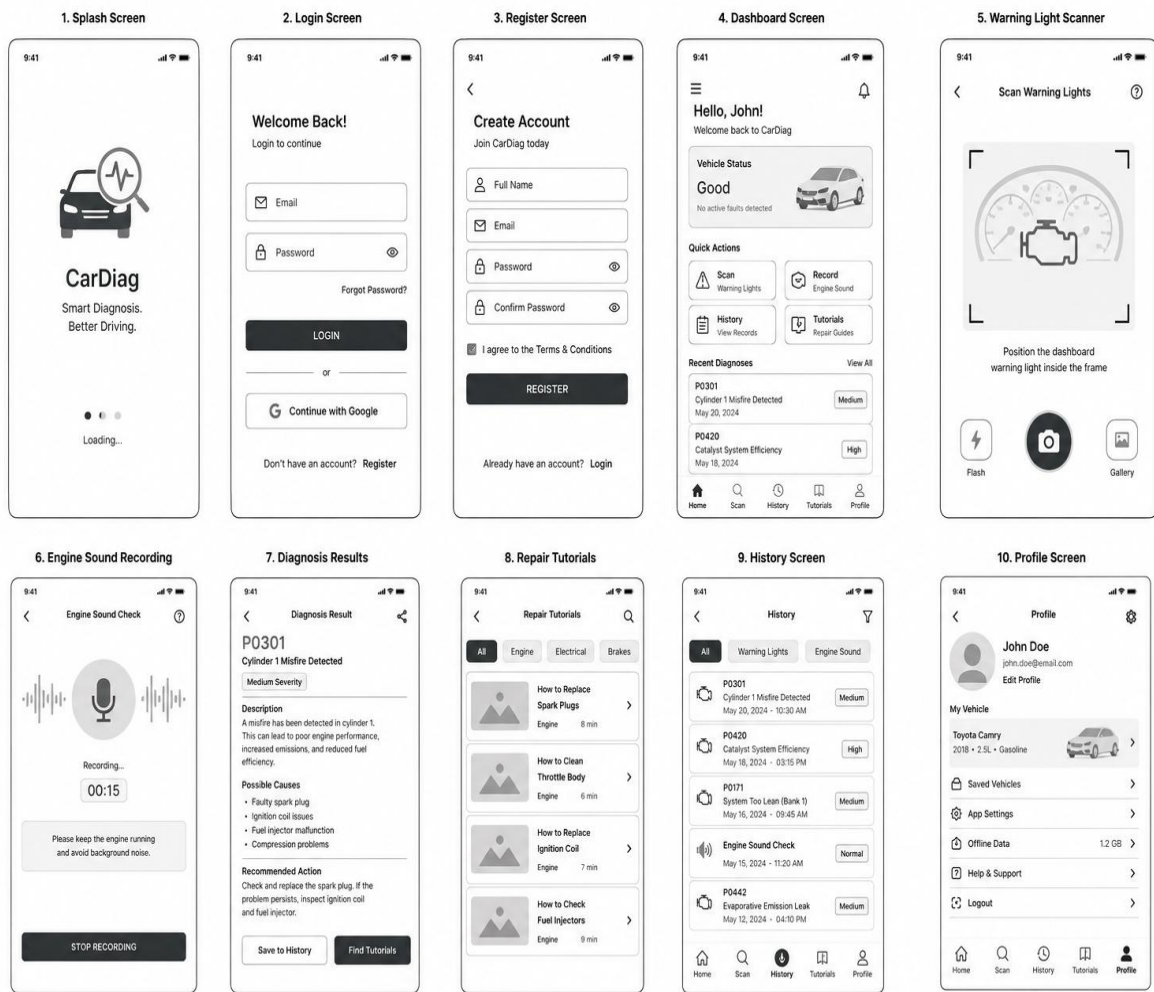


Figure 5.3 High-fidelity mockups of the ten core CarDiag screens.

The Dashboard screen greets the user, surfaces vehicle status and quick actions, and lists recent diagnoses. The Warning Light Scanner and Engine Sound Recording screens both foreground live feedback a camera viewfinder with corner markers, and a waveform with a running timer consistent with the feedback principle in Section 5.3. The Diagnosis Results screen presents the matched fault code, a severity badge, a plain-language description, possible causes, and a recommended action, with buttons to save the result to history or find related tutorials. History and Profile give the user a durable record of past diagnoses and control over their vehicle and account details.

## 5.7 Frontend Implementation

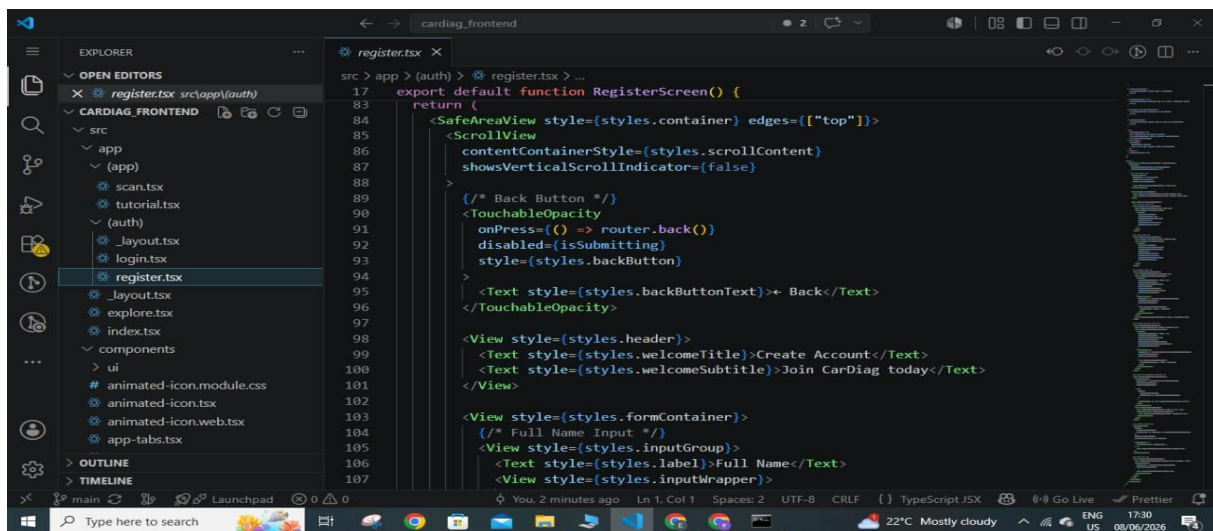
CarDiag's frontend is implemented in React Native using the CLI (rather than Expo) with the TypeScript template, chosen for static type checking and easier maintenance across a five-

person team. The project follows a feature-based folder structure under a single src directory, keeping assets, reusable components, navigation configuration, screens, API/service calls, global state, theme constants, and utility functions each in their own subfolder.

| Tool             | Version       | Purpose                                     |
|------------------|---------------|---------------------------------------------|
| Node.js          | 18.x or above | JavaScript runtime                          |
| React Native CLI | Latest        | Project initialization and build commands   |
| Android Studio   | Latest stable | Android SDK, emulator, build tools          |
| Xcode            | 14 or above   | iOS simulator and build tools (macOS only)  |
| JDK              | 17            | Required by the Android Gradle build system |

*Table 5.2 Development environment requirements for the CarDiag frontend.*

Each of the ten screens was implemented as a dedicated component under src/screens, styled against the shared theme constants defined in Section 5.4, and wired into a central navigation configuration reflecting the five-tab structure described in Section 5.5. Figure 5.4 shows an excerpt of the registration screen implementation, illustrating the functional-component and inline-styling conventions followed throughout the codebase.



*Figure 5.4 Code of the registration screen implementation in React Native.*

## 5.8 Testing

Manual testing was carried out on each implemented screen using the React Native development build on both an Android emulator and a physical Android device, verifying navigation between all five bottom-navigation tabs, form validation on the Login and Register screens, and correct rendering of severity badges on the Diagnosis Results and History screens. Camera and microphone permission prompts were verified on-device, since these cannot be reliably exercised in an emulator. Formal automated unit and integration testing of the frontend had not been completed at the time of writing and is identified in Chapter 7 as outstanding work.

## 6. DATABASE DESIGN AND IMPLEMENTATION

### 6.1 Introduction

With the requirements, system model, and interface established, the next task was to design and implement the database underpinning CarDiag. The database stores the fault knowledge base, user accounts, diagnostic records, embedding vectors, and cached video tutorials, and forms the foundation against which every diagnosis is matched. CarDiag matches user input a photo, an audio recording, or free text against a catalogue of over six thousand standardized OBD-II fault codes and returns an explanation, urgency level, repair tips, and relevant tutorials.

### 6.2 Data Elements

Data elements were identified directly from the functional requirements and grouped into five categories: user data (identity and login), fault data (the core knowledge base of OBD-II codes, categories, symptoms, causes, and repair tips), fault embedding data (the 1024-dimensional vector representation of each fault used for AI similarity search), diagnosis data (the input, matched fault, confidence score, and AI-generated report for every diagnosis performed), and video data (cached YouTube tutorial links per fault, with expiry timestamps to keep content fresh).

Table 6.3 gives a representative sample of the Fault data elements, which form the core knowledge base against which every user diagnosis is matched.

| Data element | Description                           | Data type   |
|--------------|---------------------------------------|-------------|
| id           | OBD-II fault code (e.g. P0301)        | String      |
| name         | Name of the fault                     | String      |
| category     | Fault category (Engine, Brakes, etc.) | Enum        |
| symptoms     | List of observable symptoms           | Text (JSON) |
| causes       | List of probable causes               | Text (JSON) |
| repairTips   | List of recommended repair actions    | Text (JSON) |
| urgency      | Urgency level (Critical to Low)       | Enum        |

*Table 6.1 Sample of the Fault data elements forming the core knowledge base.*

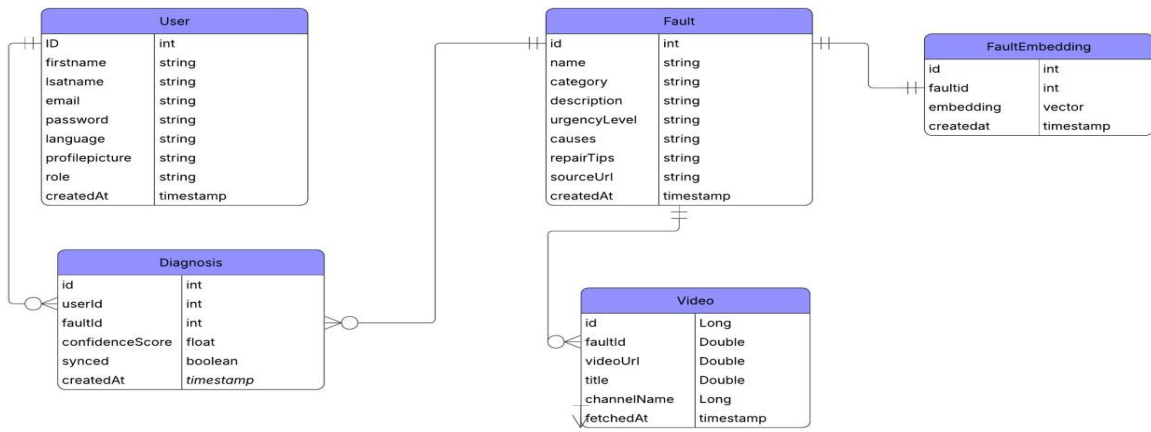
### 6.3 Conceptual Design

Five entities were identified User, Fault, FaultEmbedding, Diagnosis, and Video with Fault positioned as the central entity. A user may have many diagnoses or none, accommodating guest use; each fault has exactly one embedding, enforced by a unique constraint; a fault may be matched in many diagnoses and may have many cached videos; and deleting a fault automatically removes its embedding and cached videos through cascading deletes.

| Relationship         | Type        | Description                                                                     |
|----------------------|-------------|---------------------------------------------------------------------------------|
| User Diagnosis       | One-to-many | A user can have many diagnoses; a diagnosis belongs to one user or none (guest) |
| Fault FaultEmbedding | One-to-one  | Each fault has exactly one embedding vector                                     |
| Fault Diagnosis      | One-to-many | A fault can be matched in many diagnoses                                        |
| Fault Video          | One-to-many | A fault can have many cached tutorial videos                                    |

*Table 6.2 Entity relationships in the CarDiag conceptual data model.*

### 6.4 Entity Relationship Diagram



Type something

Figure 6.1 Entity relationship diagram of the CarDiag database.

Reading the diagram, the Fault entity sits at the centre of the design, connecting to FaultEmbedding through a one-to-one relationship and to both Diagnosis and Video through one-to-many relationships, while User connects to Diagnosis through a one-to-many relationship. This structure ensures that all diagnostic activity and supporting data revolve around the central fault catalogue.

## 6.5 Database Implementation

The database was implemented in PostgreSQL, chosen for its reliability, advanced data types, and critically its support for the pgvector extension, required to store and search the embedding vectors used by the AI component. The extension was enabled with `CREATE EXTENSION IF NOT EXISTS vector`, and the `fault_embeddings` table was defined with a dedicated `vector(1024)` column, a foreign key to the parent fault, and an `IVFFlat` index on the embedding column using the cosine-distance operator to keep similarity search fast across more than six thousand vectors.

| Property | Choice     | Reason                                        |
|----------|------------|-----------------------------------------------|
| DBMS     | PostgreSQL | Reliable, open-source, supports vector search |

|                  |                  |                                       |
|------------------|------------------|---------------------------------------|
| Vector extension | pgvector         | Stores and searches embedding vectors |
| Hosting          | Docker container | Easy and consistent local setup       |

*Table 6.3 Database management system choices and rationale.*

## 6.6 Backend Implementation

The backend is implemented in Spring Boot (Java), following a clean layered architecture consistent with the architectural principles discussed in Chapter 1, summarized in Table 6.4.

| Layer      | Responsibility                                                           |
|------------|--------------------------------------------------------------------------|
| Controller | Receives HTTP requests from the mobile application and returns responses |
| Service    | Contains business logic, including the diagnosis pipeline                |
| Repository | Communicates with the database via the JpaRepository interface           |
| Entity     | Java classes mapped to database tables via the Java Persistence API      |

*Table 6.4 Backend layer responsibilities in the CarDiag Spring Boot application.*

The diagnosis pipelines the core of the backend describes each input in text form using an external AI model, converts that description into an embedding vector, compares it against all stored fault embeddings to find the closest match, retrieves the full fault details if the match is confident enough, generates an AI report, fetches relevant video tutorials, and saves the complete diagnosis to the database.

## 6.7 Connecting the Database to the Backend

The connection is configured through Spring's datasource properties, pointing to the PostgreSQL instance and authenticating with a dedicated application user, while Hibernate is configured to validate that entity classes match the database schema on startup. Vector operations, which fall outside standard object-relational mapping, are handled through native SQL queries: float-array vectors produced by the embedding model are converted into the string format pgvector expects before storage, and the cosine-distance operator is used at query time to retrieve the most similar faults. Two one-time pipelines populated the database: the first

loaded the catalogue of 6,152 OBD-II fault codes from a structured file, and the second generated and stored an embedding vector for every fault using the chosen embedding model.

## **6.8 Chapter Summary**

This chapter has presented the complete database design and implementation for CarDiag: data elements identified from the system requirements, a conceptual design defining five entities and their relationships, an entity relationship diagram, a PostgreSQL and pgvector implementation supporting AI similarity search, a layered Spring Boot backend, and the configuration connecting the two. The resulting database provides a scalable foundation supporting fault matching, diagnosis history, video tutorials, and offline synchronization.



## 7. FINAL PROJECT PRESENTATION

### 7.1 Introduction

The final task of the CEF440 project brings together the six preceding tasks into a single account of what was built, why it was built that way, and what it demonstrates. This chapter presents CarDiag as a complete system: a walkthrough of the application as it would be demonstrated to an audience, a summary of how each of the seven tasks contributed to the finished product, an honest account of the project's limitations, and a statement of the work that remains for any future continuation.

### 7.2 Project Overview

CarDiag is an artificial-intelligence-based mobile application that helps vehicle owners in Cameroon identify car faults by scanning dashboard warning lights, recording engine sounds, or describing a problem in text. Built as a cross-platform React Native application backed by a Spring Boot and PostgreSQL/pgvector server, it matches user input against a catalogue of over six thousand OBD-II fault codes and returns a plain-language explanation, an urgency rating, recommended actions, and linked repair tutorials all designed around the specific constraints of its target users: intermittent connectivity, bilingual (English/French) communication, and limited trust in unfamiliar diagnostic technology.

### 7.3 Demonstration Walkthrough

A live demonstration of CarDiag follows the natural path of a driver encountering an unfamiliar problem, and exercises every major subsystem documented in this report.

- Onboarding the user opens the app to the splash screen, then registers or logs in; this exercises the authentication flow modelled in Sequence Diagram SD-01 and the User entity defined in Chapter 6.
- Dashboard the home screen surfaces vehicle status and quick actions, demonstrating the Dashboard process from the data flow diagram (Chapter 4) and the card-based component library from Chapter 5.
- Warning-light scan the user photographs an illuminated dashboard symbol; the app runs the capture-and-classify pipeline described in Sequence Diagram SD-02, querying the Fault entity and returning a matched code with urgency level.

- Engine sound checks as an alternative diagnostic path, the user records ten to fifteen seconds of engine audio; the same fault-matching pipeline is exercised through the audio modality rather than the image modality.
- Diagnosis result the consolidated report screen displays the fault code, severity badge, description, probable causes, and a recommended action, drawing on the FaultEmbedding similarity search implemented in Chapter 6.
- Repair tutorials the user follows through to the linked video tutorials, demonstrating the Video entity and the external YouTube API integration shown in the deployment diagram.
- History and profile finally, the user reviews their diagnosis history and vehicle profile, demonstrating the persistence of the diagnosis entity across sessions, including for guest users.

## 7.4 How the Seven Tasks Fit Together

Each task in this project built directly on the one before it, and the final application reflects the accumulation of all seven.

| Task                                  | Contribution to the finished system                                                                                |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1. Mobile app development process     | Established the technology stack: React Native frontend, layered Spring Boot backend                               |
| 2. Requirement gathering              | Produced the survey, interview, and dataset foundations that grounded every later decision                         |
| 3. Requirement analysis               | Converted raw requirements into a prioritized, validated Software Requirements Specification                       |
| 4. System modelling and design        | Produced the context, data flow, use case, sequence, class, and deployment diagrams that structured implementation |
| 5. UI design and implementation       | Defined the app's identity and delivered ten implemented screens in React Native                                   |
| 6. Database design and implementation | Delivered the PostgreSQL/pgvector schema and Spring Boot backend powering AI-based diagnosis                       |

|                               |                                                                                                         |
|-------------------------------|---------------------------------------------------------------------------------------------------------|
| 7. Final project presentation | Synthesized all preceding tasks into the complete, demonstrable CarDiag system described in this report |
|-------------------------------|---------------------------------------------------------------------------------------------------------|

*Table 7.1 Contribution of each task to the finished CarDiag system.*

This progression followed the thirteen-week course schedule set out at the start of the semester, with each two-week block culminating in a graded presentation, as shown in Table 7.2.

| Weeks   | Task                               | Marks |
|---------|------------------------------------|-------|
| 1 – 2   | Mobile app development process     | 10    |
| 3 – 4   | Requirement gathering              | 10    |
| 5 – 6   | Requirement analysis               | 10    |
| 7 – 8   | System modelling and design        | 10    |
| 9 – 10  | UI design and implementation       | 10    |
| 11 – 12 | Database design and implementation | 10    |
| 13      | Final project presentation         | 40    |

*Table 7.2 CEF440 project schedule and mark allocation by task.*

## 7.5 Achievements

- A validated, locally grounded requirement set built on an 80-respondent survey and expert interviews rather than assumption.
- A complete system model spanning context, data flow, use case, sequence, class, and deployment diagrams, each traceable back to specific requirements.
- A fully specified visual identity and ten implemented, navigable screens in a single cross-platform codebase.
- A working AI-oriented database schema combining a relational catalogue of over six thousand fault codes with vector similarity search for multimodal diagnosis.
- A hybrid offline/online architecture that responds directly to the connectivity and trust concerns raised during requirement gathering, rather than assuming ideal network conditions.

- A documented, traceable path from raw survey data through to a specific implemented screen or database field for every major feature, satisfying the course's project-based learning objective.

## **7.6 Limitations and Future Work**

As an academic project delivered within a single semester, CarDiag has known limitations that any continuation should address. The fault database, while populated with the full 6,152-entry OBD-II catalogue, has not yet been validated at scale against the specific vehicle makes and model years most common on Cameroonian roads. Full offline AI diagnosis remains a "Won't Have" for this version, in line with the MoSCoW prioritisation in Chapter 3, meaning advanced diagnosis still depends on connectivity even though basic warning-light lookup does not. The frontend testing and screen-by-screen implementation record noted in Chapter 5 was still in progress at the time of writing and would need to be completed before any store submission. Real-time mechanic chat, insurance-claim assistance, predictive maintenance, and a community forum remain explicitly out of scope for this version but were identified in Chapter 3 as valuable directions for a subsequent release.

## **7.7 Conclusion**

This report has documented the complete design and implementation of CarDiag across seven tasks: technology selection, requirement gathering, requirement analysis, system modelling, interface design, database implementation, and this final synthesis. Taken together, they demonstrate that a locally grounded, bilingual, offline-capable AI diagnostic tool is both technically achievable within the constraints of a university course and genuinely responsive to the needs, and reluctance, expressed by its intended users in Cameroon. The system delivered provides a solid and well-documented foundation for continued development beyond the scope of CEF440.

## REFERENCES

- [1] SAE International. SAE J1979 E/E Diagnostic Test Modes (OBD-II standard for diagnostic trouble codes).
- [2] International Organization for Standardization. ISO 11898 Road vehicles: Controller Area Network (CAN).
- [3] International Organization for Standardization. ISO 2575 Road vehicles: Symbols for controls, indicators and tell-tales.
- [4] Google Research. AudioSet A large-scale dataset of manually annotated audio events.
- [5] ESC-50: Dataset for Environmental Sound Classification.
- [6] Google. Material Design Guidelines. Available at: <https://m3.material.io>
- [7] Apple Inc. Human Interface Guidelines. Available at: <https://developer.apple.com/design/human-interface-guidelines>
- [8] Meta Platforms, Inc. React Native Documentation. Available at: <https://reactnative.dev/docs>
- [9] PostgreSQL Global Development Group. PostgreSQL Documentation. Available at: <https://www.postgresql.org/docs>
- [10] pgvector. Open-source vector similarity search for Postgres. Available at: <https://github.com/pgvector/pgvector>
- [11] VMware / Broadcom. Spring Boot Reference Documentation. Available at: <https://docs.spring.io/spring-boot/>
- [12] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- [13] Google Flutter Team. flutter\_bloc library documentation. Available at: <https://bloclibrary.dev>
- [14] Group 12 (2025/2026). CEF440 Task 1–6 project reports, Department of Computer Engineering, University of Buea (internal course submissions).